Automated Threat Detection and Active Response System Using Wazuh SIEM

Term Project Report

Haitem Mihoubi and Sarah Alotaibi

CS595 — Cybersecurity Fundamentals

Instructor: Professor Scott Arena

Spring 2026

Abstract

This project presents the design, deployment, and evaluation of an end-to-end automated threat detection and active response pipeline targeted at the resource constraints of small-to-medium enterprises (SMEs). The system is built on Wazuh, an open-source SIEM platform, and is deployed across two physical hosts: an Ubuntu 22.04 LTS server hosting the Wazuh Manager stack and a Windows 10 endpoint running the Wazuh Agent. A three-stage pipeline, File Integrity Monitoring on the endpoint, rule-based decision-making on the manager, and automated active response executed locally on the agent, eliminates the latency inherent in human-in-the-loop security workflows. Two response strategies are implemented and compared: a custom rule paired with a local quarantine script, and a custom rule paired with VirusTotal-based enrichment. Both are evaluated against a manual baseline using a phishing-driven EICAR test file as the attack vector. The custom rule pipeline achieves a Mean Time to Respond (MTTR) of under one second; the VirusTotal pipeline achieves 5–9 seconds with the benefit of multi-engine detection consensus; the manual baseline takes approximately 5 seconds per file but does not scale across multiple endpoints. The results demonstrate that enterprise-grade automated detection and response is achievable for SMEs without a dedicated Security Operations Center, using open-source tooling and well-documented configuration patterns.

Table of Contents

Abstract....2

Table of Contents....3

1. Introduction & Motivation....5

2. Literature Review & Background....6

2.1 Evolution of Cyber Threat Detection....6

Historical Origins: From Cryptography to Digital Defense....6

The Rise of Digital Threats: Viruses, Worms, and Malware....6

The Development of Detection Technologies....7

Advanced and Persistent Threats in the Modern Era....7

The Role of Artificial Intelligence in Modern Threat Detection....7

The Continuing Arms Race....7

2.2 The Role of SIEM in Modern Security Operations Centers (SOC)....8

2.3 Introduction to Wazuh Architecture....9

The Wazuh Agent....9

The Wazuh Agent Modules and Communication Protocols....10

The Wazuh Server....10

The Wazuh Indexer....11

The Wazuh Dashboard....11

Deployment Models....11

Wazuh Cyber Threat Intelligence (CTI)....11

2.4 Active Response vs. Passive Monitoring....12

The Limitations of Passive Monitoring....12

The Case for Active Response....12

Types of Active Response in Wazuh....13

Practical Applications of Active Response....13

Custom Active Response....13

2.5 SIEM Platform Options and the Rationale for Choosing Wazuh....14

Overview of Available SIEM Solutions....14

Commercial Enterprise Platforms....14

Cloud-Native Platforms....14

Open-Source Alternatives....15

Why Wazuh Was Selected....15
2.6 True Positives, False Positives, True Negatives, and False Negatives in Cybersecurity Detection 15
The Four Outcomes Defined....16
3. Security Model and Proposed Solution....16
3.1 Security Model....16
Threat Model....16
Trust Model....17
3.2 System Overview....17
Two-Node Architecture....18
Active Response Pipeline Logic....18
4. Implementation....19
4.1 Wazuh Installation....19
4.2 Detecting and Removing Malware Using Custom Scripts and Rules....27
4.3 Detecting and Removing Malware Using VirusTotal Integration....29
5. Evaluation & Results....34
5.1 Methodology....34
5.2 Attack Scenario — Phishing-Driven Malware Download....34
5.3 Results: Custom Rules and Scripts....36
5.4 Results: VirusTotal Integration....37
5.5 Comparative Analysis....37
6. Discussion....38
6.1 Limitations....38
6.2 Future Work....39
7. Conclusion....39
References....40

1. Introduction & Motivation

The modern cybersecurity landscape exposes a growing and increasingly serious challenge for small-to-medium enterprises. While large organizations can sustain the operational cost of a 24/7 Security Operations Center (SOC), staffed with trained analysts, fed by commercial threat intelligence, and backed by enterprise-grade tooling, the majority of businesses worldwide operate without any equivalent capability. This resource gap is a structural vulnerability that adversaries actively exploit.

Ransomware, one of the most prevalent and damaging threat categories facing those enterprises today, is capable of encrypting an entire file system within minutes of execution. In that window, a human analyst following a manual triage workflow has no chance of intervening before irreversible damage occurs. The bottleneck is not skill or intent, it is the latency inherent in human-in-the-loop security response.

This project is motivated by two converging observations. First, the tools required to automate meaningful threat detection and response are no longer the exclusive domain of well-funded enterprises. Open-source platforms such as Wazuh have matured to the point where they offer capabilities, such as File Integrity Monitoring, rule-based alerting, and native active response, that are functionally comparable to commercial alternatives such as Splunk or IBM QRadar, at a fraction of the cost. Second, the primary barrier for SMEs is the absence of a configured, tested, and operational deployment of that tooling.

This project bridges that gap. The central task is the implementation, configuration, and evaluation of an end-to-end automated threat detection and active response pipeline, built on an open-source solution and targeted specifically at the resource constraints of small-to-medium enterprises. The system is built around Wazuh, deployed in a two-node architecture consisting of a Wazuh Manager running on Ubuntu 22.04 LTS and a Wazuh Agent installed on a Windows 10 endpoint. It operates through a three-stage pipeline: continuous File Integrity Monitoring on the endpoint, rule-based detection and decision-making on the Manager, and automated active response executed locally on the agent without any human intervention.

This report is organized as follows. Section 2 provides background on the SOC and SIEM landscape, introducing Wazuh architecture, passive and active response. Section 3 describes the Security Model and presents the Proposed Solution. Section 4 documents the implementation in detail. Section 5 presents the evaluation methodology and results. Section 6 discusses limitations and directions for future work, and Section 7 concludes.

2. Literature Review & Background

2.1 Evolution of Cyber Threat Detection

The field of cybersecurity and threat detection has changed dramatically since its origins. What began as rudimentary manual defenses has grown into a sophisticated, technology-driven discipline that underpins the security of modern society. Tracing this journey is important for understanding why automated systems such as the one proposed in this project are now considered essential rather than optional.

Historical Origins: From Cryptography to Digital Defense

The roots of cybersecurity stretch back well before the modern internet existed. According to Zaid and Garai (2024), some of the earliest forms of information protection emerged during World War II, when nations deployed advanced encryption machines such as the Enigma to secure military communications. The effort to decode these ciphers, led by teams at Bletchley Park, planted the intellectual seeds of what would eventually become the modern cybersecurity field. Cryptography served as the first serious line of defense against adversaries seeking to intercept secrets, establishing a pattern of offense-versus-defense that persists to this day.

The Rise of Digital Threats: Viruses, Worms, and Malware

As computing moved from isolated machines to interconnected networks, the nature of threats changed fundamentally. The early days of networked computing brought the first major incidents, the Morris worm of 1988 being a landmark example, but these were isolated events compared to what followed. As the internet grew in reach, so did the ambitions of those exploiting it. Hackers motivated by curiosity, political goals, or financial gain began targeting operating systems, websites, and human behavior. Malware, worms, and viruses proliferated rapidly, and organized cybercrime syndicates emerged, driving large-scale data breaches and identity theft campaigns on an unprecedented scale (Zaid & Garai, 2024).

The Development of Detection Technologies

The cybersecurity community responded to these growing threats by developing increasingly capable defensive tools. Antivirus software and firewalls became standard components of organizational security, and intrusion detection systems were introduced to provide deeper network visibility. Governments began establishing dedicated cybersecurity agencies, and international cooperation frameworks were created to coordinate responses to cross-border cybercrime. By the early 2000s, cybersecurity had grown into a multi-billion-dollar global industry, employing specialists ranging from ethical hackers and malware analysts to network engineers and incident response professionals.

Advanced and Persistent Threats in the Modern Era

The evolution of threats did not plateau with the arrival of antivirus software and firewalls. More sophisticated attack types emerged that exposed the limits of traditional detection approaches. Supply chain attacks, for instance, involve infiltrating a trusted third-party vendor to reach a more heavily defended target, a strategy used to devastating effect in the 2020 SolarWinds breach, which compromised thousands of organizations through a single corrupted software update. Advanced Persistent Threats (APTs), characterized by prolonged, stealthy network intrusions, were illustrated in 2015 when attackers accessed U.S. Office of Personnel Management systems and extracted data on over 21 million individuals before being detected. Ransomware escalated in 2017 with WannaCry, which spread to over 300,000 systems in 150 countries by exploiting Windows vulnerability.

The Role of Artificial Intelligence in Modern Threat Detection

The most significant shift in threat detection over the past decade has been the integration of artificial intelligence (AI) and machine learning (ML). Zaid and Garai (2024) describe these technologies as capable of analyzing vast streams of data in real time, monitoring network traffic, login behavior, and system logs simultaneously to detect subtle anomalies that human analysts would almost certainly miss. Unlike rule-based systems, AI-driven tools can learn from past incidents and adapt to new attack patterns, making them effective even against zero-day threats for which no prior signatures exist. When a threat is confirmed, AI-powered systems can isolate infected endpoints, trigger remediation scripts, and generate forensic reports automatically, capabilities that are central to the design of the proposed system in this project.

The Continuing Arms Race

Despite advances in detection capability, the threat environment continues to grow in complexity and scale. Attackers now leverage the same AI tools available to defenders, using them to craft more convincing phishing campaigns, automate vulnerability discovery, and evade detection. Emerging attack surfaces, including IoT devices, deepfake technology, cloud infrastructure, and insider threats, demand continuous adaptation from security teams. Zaid and Garai (2024) note that ransomware, social engineering, and supply chain attacks now target not just commercial organizations but critical infrastructure, healthcare systems, and

democratic processes. The stakes have never been higher, and the gap between the speed of modern attacks and the speed of manual human response has never been wider: a gap that automated detection and response systems are specifically designed to close.

The trajectory traced above establishes why the proposed Wazuh SIEM-based system is needed: each generation of threats has outpaced the detection methods that preceded it, and the current era demands proactive, automated, and continuously learning defenses.

2.2 The Role of SIEM in Modern Security Operations Centers (SOC)

A Security Operations Center (SOC) is a centralized function or team responsible for improving an organization's cybersecurity posture by preventing, detecting, and responding to threats. SOC teams monitor identities, endpoints, servers, databases, network applications, websites, and other systems to uncover potential cyberattacks in real time, while also performing proactive security work using the latest threat intelligence to identify and address vulnerabilities before attackers can exploit them. Most SOCs operate around the clock, seven days a week, ensuring continuous protection across the organization's entire environment (Microsoft Security, n.d.).

Within this structure, Security Information and Event Management (SIEM) is one of the most critical tools available to SOC. Without it, continuous monitoring and threat detection at scale would be extremely difficult to sustain. Modern SIEM solutions equip SOC teams with four core capabilities: log aggregation, contextual correlation, alert reduction, and automated response.

Log aggregation is the foundation of SIEM's value. The system collects and correlates log data from every endpoint, operating system, virtual machine, application, and network event across the organization, establishing a baseline of normal activity from which anomalies, potential indicators of malware, ransomware, or unauthorized access, can be identified. Beyond raw collection, SIEM adds context by linking individual incidents together, allowing analysts to detect multi-stage attacks that would otherwise appear as unrelated, benign events.

Alert management is equally vital. The volume of alerts generated in a modern enterprise environment can easily overwhelm an analyst team. By applying analytics and AI-driven correlation, SIEM filters the alert stream down to the subset most likely to represent genuine threats, reducing the time analysts waste on false positives and ensuring that critical events receive the attention they deserve.

Automated response represents perhaps the most operationally significant advancement in modern SIEM capability, and it is directly relevant to this project. Built-in detection rules allow SIEM platforms to identify probable threats and trigger containment actions, blocking network traffic, isolating endpoints, or locking

accounts, without requiring human intervention at every step. This capability is central to the Automated Threat Detection and Active Response System using Wazuh SIEM proposed here, where minimizing the time between detection and containment is a primary objective.

SIEM also supports broader SOC functions beyond threat detection, including threat intelligence integration, compliance management, incident response coordination, and post-incident root cause analysis. Together, these capabilities ensure that an organization not only responds to threats as they arise but continuously refines its defenses to address future risks.

Taken together, SIEM is the analytical and operational backbone of the modern SOC, not a peripheral tool but the central platform through which security teams function effectively at scale. The growing sophistication of cyber threats makes this role more important with each passing year.

2.3 Introduction to Wazuh Architecture

Wazuh is an open-source security platform that functions as both a Security Information and Event Management (SIEM) system and an Extended Detection and Response (XDR) solution. Its architecture is composed of a multi-platform Wazuh agent and three central components: the Wazuh server, the Wazuh indexer, and the Wazuh dashboard. These components form an integrated data pipeline in which security events are collected, analyzed, stored, and visualized to support real-time threat detection and response.

The Wazuh Agent

The Wazuh agent serves as the data collection layer of the architecture, deployed directly on monitored endpoints. It continuously sends events to the Wazuh server for analysis, establishing a connection over TCP port 1514 by default and using AES encryption with 128-bit blocks and 256-bit keys to protect all communications in transit. The agent supports a broad range of operating systems, including Linux, Windows, macOS, Solaris, and AIX, making it suitable for heterogeneous enterprise environments.

For systems and devices where agent installation is not possible, Wazuh also supports agentless monitoring. Network devices such as firewalls, switches, routers, and access points can forward log data via Syslog and SSH, extending the platform's visibility across the full network infrastructure without requiring software to be installed on every device.

The Wazuh Agent Modules and Communication Protocols

The Wazuh agent operates on a modular architecture in which each module handles a distinct set of security tasks independently. This design allows administrators to

configure and enable only the capabilities their environment requires. The key agent modules are described below (Wazuh, Inc., n.d.-b):

- Log Collector: Reads flat log files and Windows events, supporting XPath filters and multi-line formats such as Linux Audit logs.
- Command Execution: Runs authorized commands periodically and reports output back to the Wazuh server for further analysis.
- File Integrity Monitoring (FIM): Monitors the file system and reports file creation, deletion, or modification in real time, capturing who, what, and when details.
- Security Configuration Assessment (SCA): Provides continuous configuration assessment using CIS benchmark checks, with support for custom security policies.
- System Inventory: Periodically collects data on operating system version, running processes, installed applications, network interfaces, and open ports.
- Malware Detection: Uses a non-signature-based approach to detect rootkits, hidden processes, hidden files, and hidden ports.
- Active Response: Executes automatic containment actions when threats are detected, such as blocking a network connection, terminating a process, or deleting a malicious file.
- Container Security Monitoring: Integrates with the Docker Engine API to detect changes to container images, network configurations, and data volumes.
- Cloud Security Monitoring: Communicates natively with cloud provider APIs (AWS, Azure, GCP) to detect infrastructure changes and collect cloud service logs.

Communication between the agent and the Wazuh server takes place over a secure TCP or UDP channel with real-time data encryption and compression. The channel includes flow control mechanisms to protect network bandwidth and prevent flooding. Before an agent connects to the server for the first time, it must complete an enrollment process that generates a unique authentication key used for all subsequent encrypted communications. Once enrolled, agents can be remotely upgraded, monitored, and reconfigured from the Wazuh server.

The Wazuh Server

The Wazuh server is the analytical core of the platform. Upon receiving events from connected agents, it decodes and runs them through its rule-matching Analysis engine, which processes security events to identify threats, anomalies, and compliance violations in real time. The server also provides centralized agent management, allowing administrators to push configuration updates and upgrades remotely across all monitored endpoints.

After processing, the Wazuh server uses Filebeat to forward alert and event data to the Wazuh indexer over a TLS-encrypted channel, with the indexer listening on port 9200/TCP by default. This encrypted pipeline ensures that security data is protected at every stage of its journey from endpoint to storage.

The Wazuh Indexer

The Wazuh indexer is a highly scalable, full-text search and analytics engine responsible for indexing and storing all alerts and events produced by the Wazuh server. It functions as the platform's data repository, enabling fast querying and retrieval of historical security records. Depending on the size and availability requirements of the deployment, the indexer can be configured as either a single-node instance or a multi-node cluster.

The Wazuh Dashboard

The Wazuh dashboard is the web-based interface through which security analysts interact with the platform. It queries the Wazuh server API listening on port 55000/TCP by default to display configuration and status information for the server and all connected agents. All communication between the dashboard and the server API is secured with TLS and authenticated with credentials. The dashboard also queries the Wazuh indexer directly to render real-time views of alerts, threat trends, and compliance reports.

Deployment Models

Wazuh supports three deployment configurations to accommodate environments of different sizes and availability requirements. An all-in-one deployment installs all components on a single host, which is suitable for lab environments and small-scale deployments. A single-node deployment places each component on a dedicated server, recommended for medium-sized environments requiring higher performance. For large environments with high event throughput or requirements for fault tolerance and high availability, a multi-node deployment uses clusters of Wazuh servers and indexers managed behind a load balancer.

Wazuh Cyber Threat Intelligence (CTI)

A distinctive feature of the Wazuh architecture is its integrated Cyber Threat Intelligence (CTI) service, a publicly accessible platform that collects, analyzes, and disseminates information on emerging vulnerabilities. It delivers timely updates on Common Vulnerabilities and Exposures (CVEs), severity scores, exploitability assessments, and mitigation strategies by aggregating data from operating system vendors and major vulnerability databases. This service integrates directly with the Wazuh Vulnerability Detection module, enriching alerts with up-to-date threat context.

Collectively, these components make Wazuh a comprehensive, modular, and scalable platform for automated threat detection and active response, capabilities that are at the core of the system proposed in this project.

2.4 Active Response vs. Passive Monitoring

In cybersecurity, the difference between active response and passive monitoring is not merely a matter of degree, it represents a fundamental distinction in philosophy. Passive monitoring observes, logs, and alerts; it relies on human analysts to act on what it finds. Active response goes further: when a qualifying threat is detected, the system acts autonomously to contain or neutralize it, without waiting for manual instruction. This distinction is central to understanding the value of the system proposed in this project.

The Limitations of Passive Monitoring

Passive monitoring has historically formed the foundation of SIEM-based security. In this model, the platform collects log data, generates alerts, and surfaces findings to analysts through dashboards and reports. While this provides valuable visibility, it leaves the organization's actual response time entirely dependent on analyst availability and judgment. According to Wazuh, Inc. (n.d.-c), security teams routinely struggle to address high-severity events quickly, provide complete mitigation actions, and collect the real-time contextual data needed to understand an incident's full scope. These difficulties are especially acute in resource-constrained environments, where a high volume of daily alerts makes it easy for critical events to be deprioritized or missed entirely.

The Case for Active Response

Active response was developed specifically to overcome the speed and consistency limitations of human-dependent monitoring. Rather than waiting for an analyst to investigate an alert, the Wazuh SIEM and XDR platform can execute predefined response actions the moment a qualifying alert is triggered, providing real-time threat containment, reducing alert fatigue, and ensuring that high-priority incidents are handled immediately and consistently. This is particularly valuable in environments where security teams are small relative to the volume of activity they must monitor.

The Wazuh Active Response module works by executing scripts on monitored endpoints when an alert matching a specific rule ID, severity level, or rule group is generated. Because these responses are automated and rule-driven, they are applied uniformly regardless of time of day, analyst workload, or organizational size.

Types of Active Response in Wazuh

Wazuh implements two categories of active response. Stateless responses are one-time actions with no built-in reversal mechanism. for example, permanently removing a malicious file. Stateful responses, by contrast, are designed to revert automatically after a specified period, for example, temporarily blocking an IP address for 180

seconds following a brute-force attempt before restoring normal access. This two-tier model gives administrators fine-grained control over the aggressiveness and reversibility of automated actions.

Practical Applications of Active Response

The Wazuh documentation provides several concrete illustrations of active response in practice. When malicious files are detected through the File Integrity Monitoring module and VirusTotal integration, the Active Response module can automatically remove them from the endpoint, closing the gap between detection and remediation. Against Denial-of-Service (DoS) attacks, the module can block attacking IP addresses at the firewall level, preserving the availability of targeted services. In brute-force scenarios, the module can temporarily disable the targeted user account once a threshold of failed login attempts is crossed, preventing password-guessing attacks from succeeding without permanently locking out legitimate users.

Custom Active Response

Beyond the library of built-in response scripts, the Wazuh Active Response module is designed to be extended. Security teams can write custom response scripts in any programming language, allowing responses to be tailored to the specific threats, workflows, and compliance requirements of their organization. This extensibility ensures that as the threat landscape evolves, the response capability can evolve alongside it without requiring a change in the underlying platform.

Passive monitoring will always play a role in security operations, it provides the audit trail, the forensic record, and the situational awareness that informed decision-making requires. But in an era where attacks unfold in minutes or seconds, passive observation alone is no longer sufficient. Active response, as implemented in Wazuh SIEM, bridges the gap between detection and containment, and it is this bridge that the proposed system in this project is designed to build and demonstrate.

2.5 SIEM Platform Options and the Rationale for Choosing Wazuh

Overview of Available SIEM Solutions

Before choosing a platform for an automated threat detection and active response system, it is important to survey the SIEM solutions available to practitioners. The market broadly divides into three categories: commercial enterprise platforms, cloud-native SIEMs, and open-source alternatives (Unihackers, n.d.).

Commercial Enterprise Platforms

The most established commercial offerings include IBM QRadar, Splunk Enterprise Security, and Microsoft Sentinel. IBM QRadar is widely deployed in large enterprises and provides powerful correlation capabilities, but it carries substantial licensing costs and requires dedicated infrastructure expertise to operate effectively. Splunk Enterprise Security is known for its flexible query language and vast ecosystem of integrations, but its pricing model, based on data ingestion volume, makes it expensive for smaller organizations or research environments. Both platforms are feature-rich but create high barriers to entry for academic projects, start-ups, and small organizations.

Cloud-Native Platforms

Microsoft Sentinel and Google Chronicle represent the newer generation of cloud-native SIEMs. Microsoft Sentinel integrates tightly with the Azure ecosystem and offers strong AI-driven analytics, but its pay-as-you-go pricing scales rapidly with data volume, and it assumes a cloud-first infrastructure that may not reflect the on-premises or hybrid environments where many real-world threats are most acute (Craig, 2023). Chronicle, similarly, is purpose-built for Google Cloud environments and offers impressive telemetry at scale, but again at significant cost and with limited flexibility outside its native cloud context.

Open-Source Alternatives

The open-source category includes platforms such as the ELK Stack (Elasticsearch, Logstash, Kibana), OSSEC, and Wazuh. The ELK Stack provides a powerful foundation for log ingestion and visualization but requires significant custom development to build the rule-based detection and active response capabilities that come out of the box with a dedicated SIEM. OSSEC, the predecessor from which Wazuh was forked, offers host-based intrusion detection but lacks the modern dashboard, scalable indexer, and extended detection and response (XDR) functionality that Wazuh has since introduced (FMI Security, n.d.).

Why Wazuh Was Selected

Wazuh was chosen for this project on the basis of four primary criteria: cost, feature completeness, extensibility, and suitability for a research and demonstration environment.

Cost and Accessibility. Wazuh is fully open-source and free to deploy, making it accessible without institutional licensing agreements or cloud spend commitments.

Feature Completeness Out of the Box. Unlike the ELK Stack, which requires significant additional engineering to become a functional SIEM, Wazuh ships with an integrated rule engine, File Integrity Monitoring, vulnerability detection, Security Configuration Assessment, compliance reporting, and a built-in Active Response module.

Extensibility. Wazuh's modular architecture and support for custom active response scripts in any programming language allow the system to be adapted to novel threats and organizational workflows without replacing the platform.

Community and Documentation. Wazuh benefits from an active open-source community and comprehensive official documentation, reducing the practical barriers to deployment and configuration that would otherwise consume significant project time.

2.6 True Positives, False Positives, True Negatives, and False Negatives in Cybersecurity Detection

A fundamental challenge in any threat detection system is to accurately detect threats. Every alert a SIEM generates falls into one of four categories derived from the confusion matrix (V500 Systems, n.d.):

The Four Outcomes Defined

True Positive (TP): The system correctly identifies a real threat.

False Positive (FP): The system raises an alert for an event that is not actually a threat.

True Negative (TN): The system correctly produces no alert for a benign event.

False Negative (FN): The system fails to detect a real threat.

3. Security Model and Proposed Solution

3.1 Security Model

Threat Model

The threat model for this system is grounded in the STRIDE framework, which categorizes adversarial behavior into Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The adversary is modeled as an external attacker who has already achieved initial access to a monitored Windows 10 endpoint, for example, through a phishing payload or a drive-by download. Modern intrusion scenarios rarely begin at zero; the critical detection window lies between initial access and the execution of a damaging payload.

Within this assumption, the system is designed to detect and respond to three concrete attack categories, each of which maps onto one or more STRIDE threats:

Malware injection: the adversary drops an unauthorized executable into a sensitive or system-privileged directory (e.g., the Sensitive Folder monitored by FIM, or a Windows system path). This corresponds primarily to **Tampering** with the file system, and, when the payload escalates rights, to **Elevation of Privilege**.

File integrity violations: the adversary modifies critical system files in order to establish persistence, disable logging, or alter security configurations. This is a textbook case of **Tampering** and, where it disables audit trails, also **Repudiation**.

Brute-force access attempts: the adversary uses repeated credential-stuffing or dictionary attacks over RDP or SSH to move laterally or gain administrative control. This combines **Spoofing** (impersonation of a legitimate user) with **Elevation of Privilege**, and at high volumes also produces a **Denial of Service** condition against the targeted account.

Threats outside this scope, including pure network-layer attacks against unmonitored hosts, attacks that compromise the Manager itself, and adversaries who succeed in disabling the agent before triggering any FIM event, are explicitly out of scope and are revisited as limitations in Section 6.1.

Trust Model

The trust model assigns explicit levels of trust to each component of the two-node architecture.

The **Wazuh Manager** is designated as the *Root of Trust*. It is assumed to be hardened, and free from compromise. All detection logic, rule evaluation, and active response orchestration originates from this node. Its integrity is a prerequisite for the

correctness of the entire system: if the Manager is compromised, the security guarantees of the pipeline collapse entirely.

The **Wazuh Agent** is treated as a *Managed Entity*, a node that is monitored and directed by the Manager, but which is not inherently trusted. Given that the threat model begins with an adversary who already has initial access to this endpoint, it cannot be assumed to be free from compromise during active scenarios.

All communication between the Manager and Agent is encrypted using TLS, ensuring that log data forwarded from the endpoint and response commands pushed back to it cannot be intercepted or tampered with in transit. This encryption boundary means the integrity of the detection-response cycle is maintained even in the presence of a network-level adversary.

3.2 System Overview

The proposed solution is an end-to-end, automated threat detection and active response pipeline built entirely on open-source tooling, designed for deployment by organizations without a dedicated Security Operations Center. At its core, the system leverages **Wazuh,** an open-source SIEM platform that provides integrated log analysis, File Integrity Monitoring (FIM), rule-based alerting, and a native active response module, to eliminate the latency inherent in human-in-the-loop security workflows.

The key design principle is **automation at the response layer**. Traditional SIEM deployments surface alerts to a dashboard and rely on a human analyst to act. This system removes that dependency: when a high-severity rule is matched, the response is triggered automatically, without human intervention. The primary performance metric this enables is a measurable reduction in **Mean Time to Respond (MTTR)** the elapsed time between the creation of a threat artifact and its neutralization.

The system is deployed in a controlled lab environment using native operating system installations (not virtual machines) to ensure realistic performance benchmarks and reproducible results.

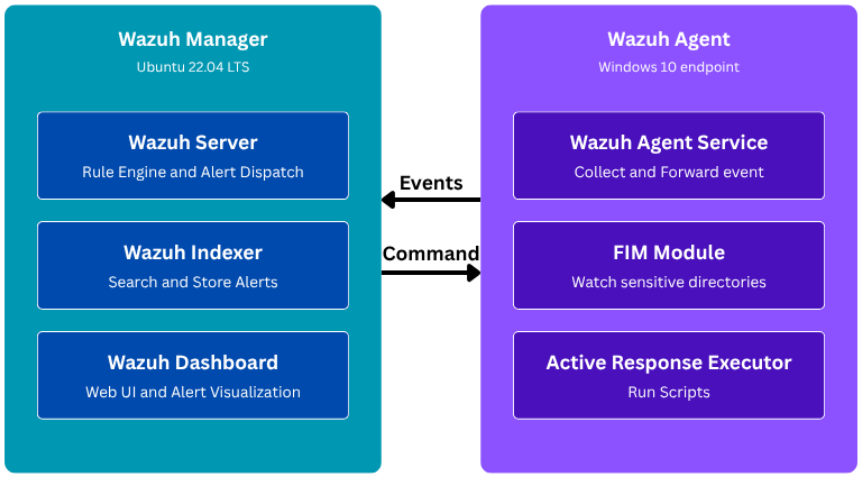
Two-Node Architecture

The physical architecture consists of two nodes connected over a private network:

Node 1: Wazuh Manager (Ubuntu 22.04 LTS). This node runs the full Wazuh server stack, including the Wazuh Manager (Server), Wazuh Indexer (based on

OpenSearch), and the Wazuh Dashboard. It is responsible for ingesting log data from enrolled agents, evaluating events against the rule engine, generating alerts, and dispatching active response commands. All detection logic resides here.

Node 2: Wazuh Agent (Windows 10). This node represents the monitored endpoint. The Wazuh Agent runs as a background service and performs real-time monitoring of selected file paths, system logs, and process activity. Collected events are forwarded over an encrypted channel to the Manager. Upon receiving an active response command from the Manager, the Agent executes the corresponding local script to remediate the detected threat.



Wazuh Nodes and Components

Active Response Pipeline Logic

The system operates through a sequential three-stage pipeline:

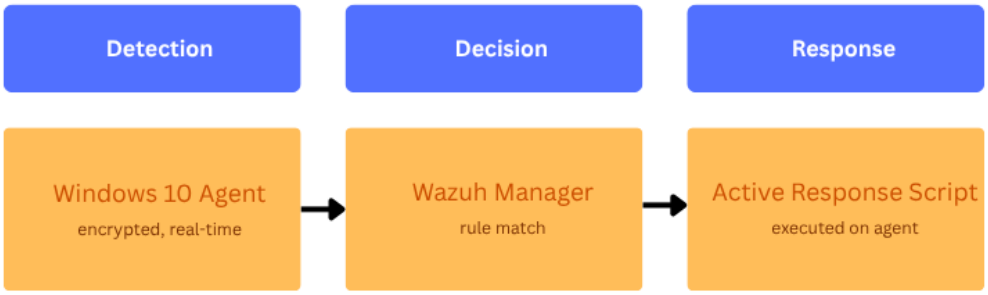
Stage 1: Detection. The Wazuh Agent on the Windows 10 endpoint monitors a designated *Sensitive Folder* using real-time File Integrity Monitoring. FIM computes and records cryptographic hashes, file sizes, timestamps, and permissions for all files within the monitored path. Any unauthorized creation, modification, or deletion of a file generates an FIM event, and forwards it to the Wazuh Manager.

Stage 2: Decision. The Wazuh Manager rules engine evaluates incoming events against a custom XML rule-set. Each rule specifies a pattern and an associated severity level. When an event matches a rule at or above the configured severity threshold, an alert is generated and the corresponding active response action is triggered automatically, with no human approval step.

Stage 3: Response. Upon triggering, the Manager dispatches an Active Response command to the agent on the Windows endpoint. The agent executes a script and

takes action against the offending file and, where applicable, terminates the associated process.

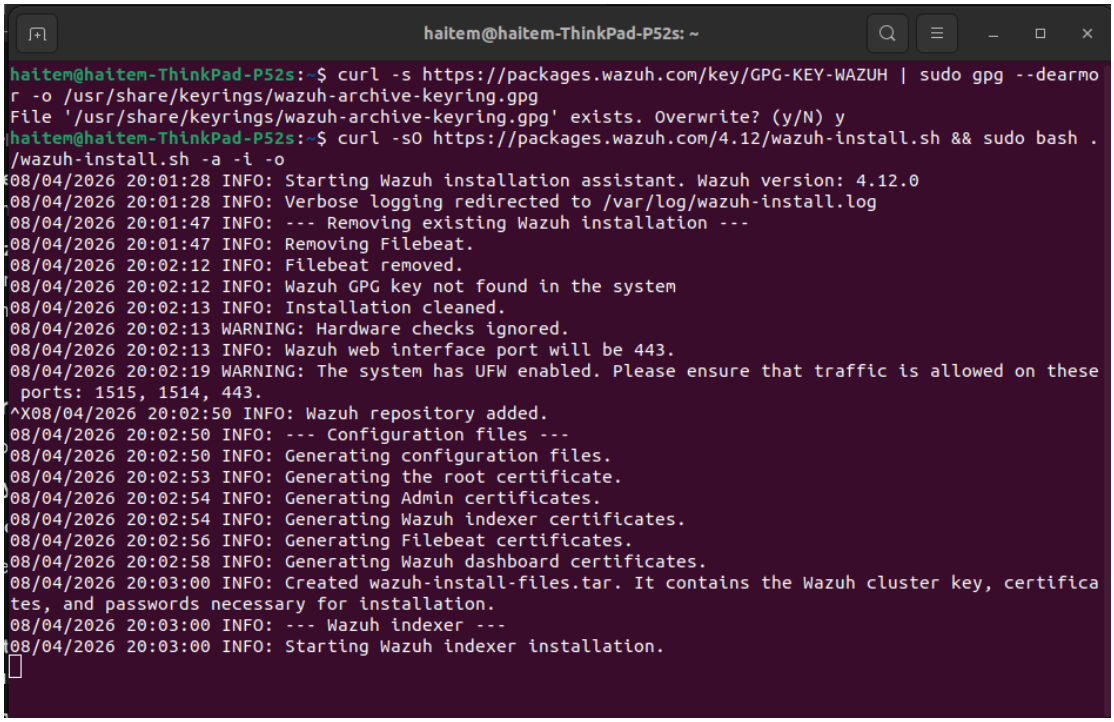
The complete pipeline can be summarized as:



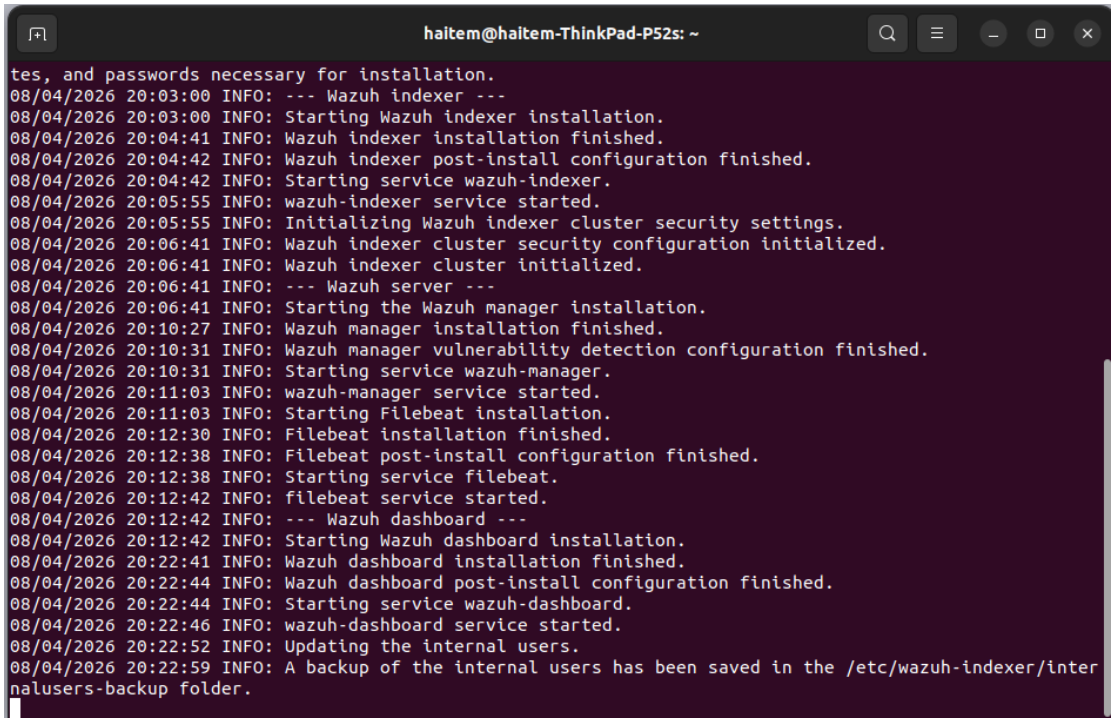
4. Implementation

4.1 Wazuh Installation

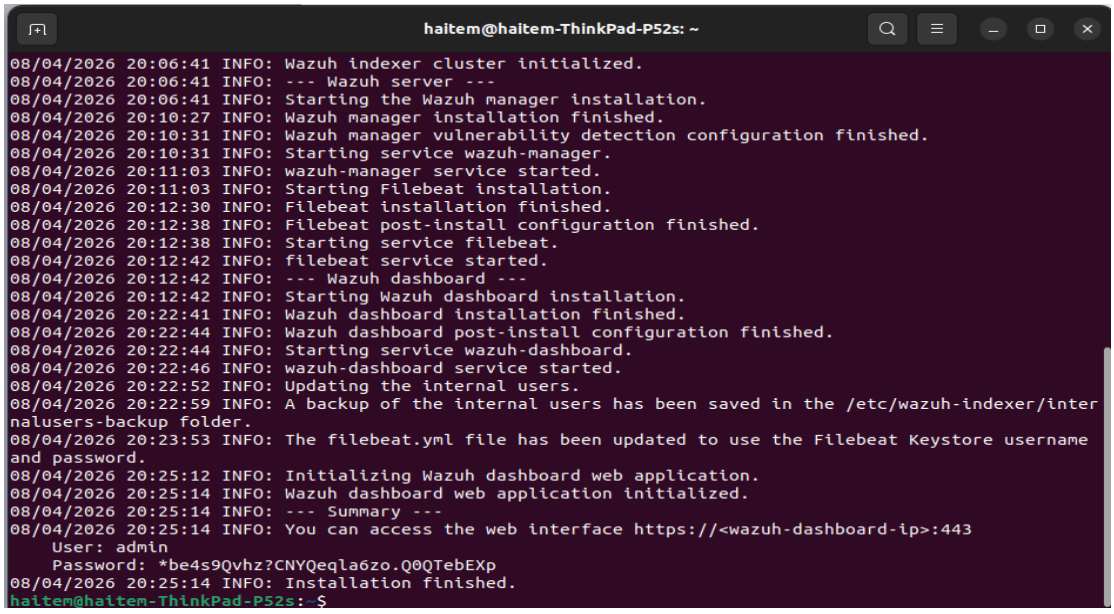
For a proper Wazuh installation we first get gpg key from repository



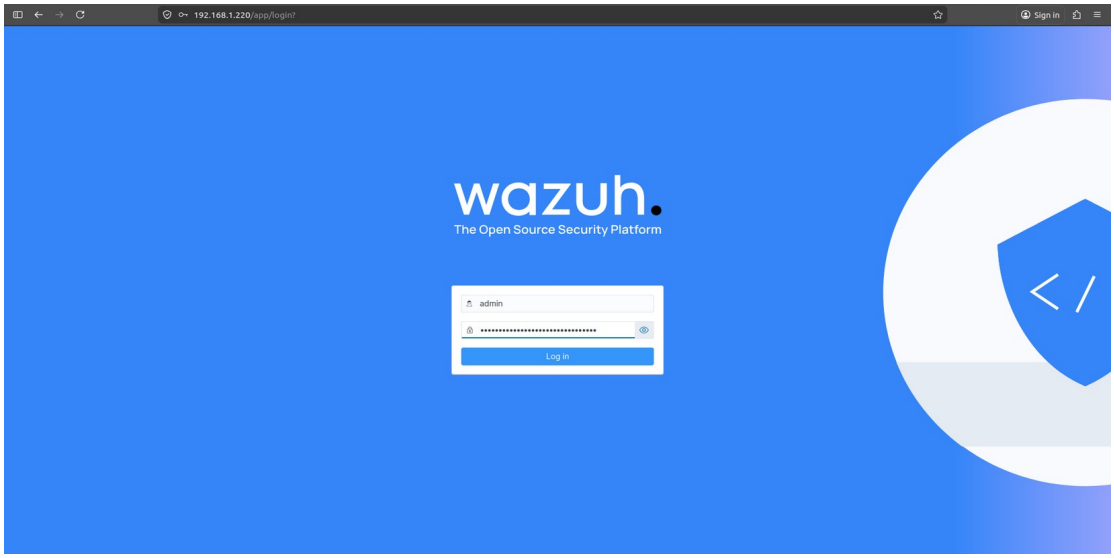
Then we need to install wazuh Components: Indexer, Server and dashboard, we can install them separately or with a single command using the -a argument for all



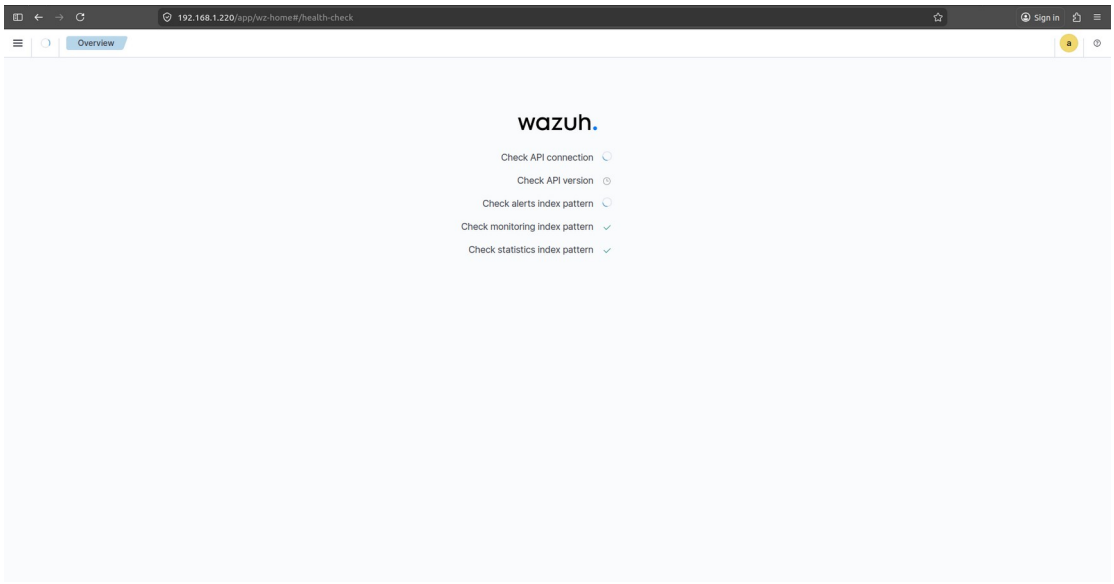
At the end, Wazuh will provide the User and Password to access the platform, we can update the credentials any time in the settings but we save them for now.

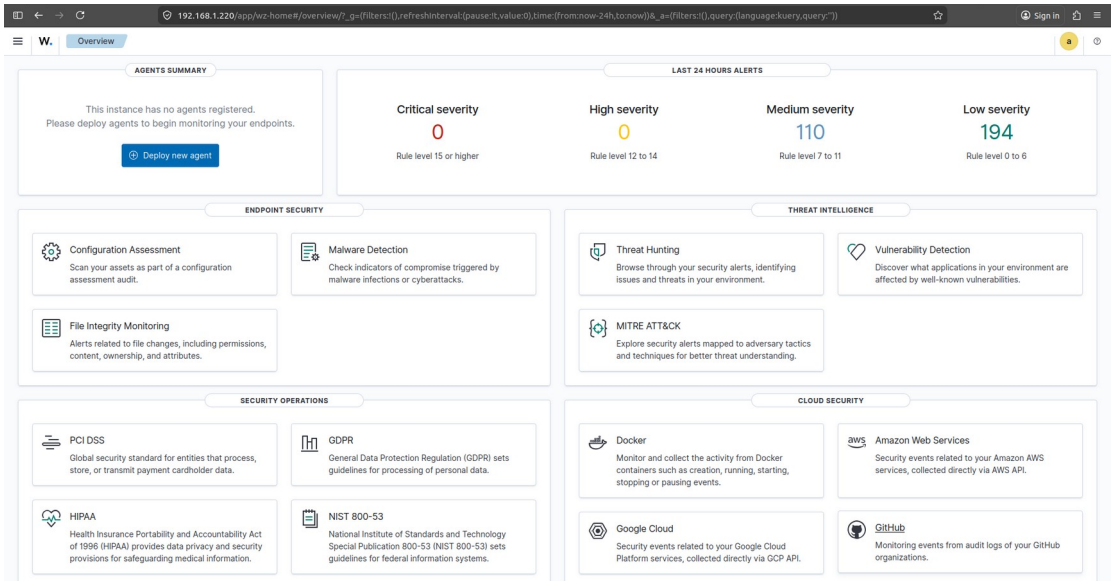


To access this login, we use this link https://<IP-ADDRESS>:443

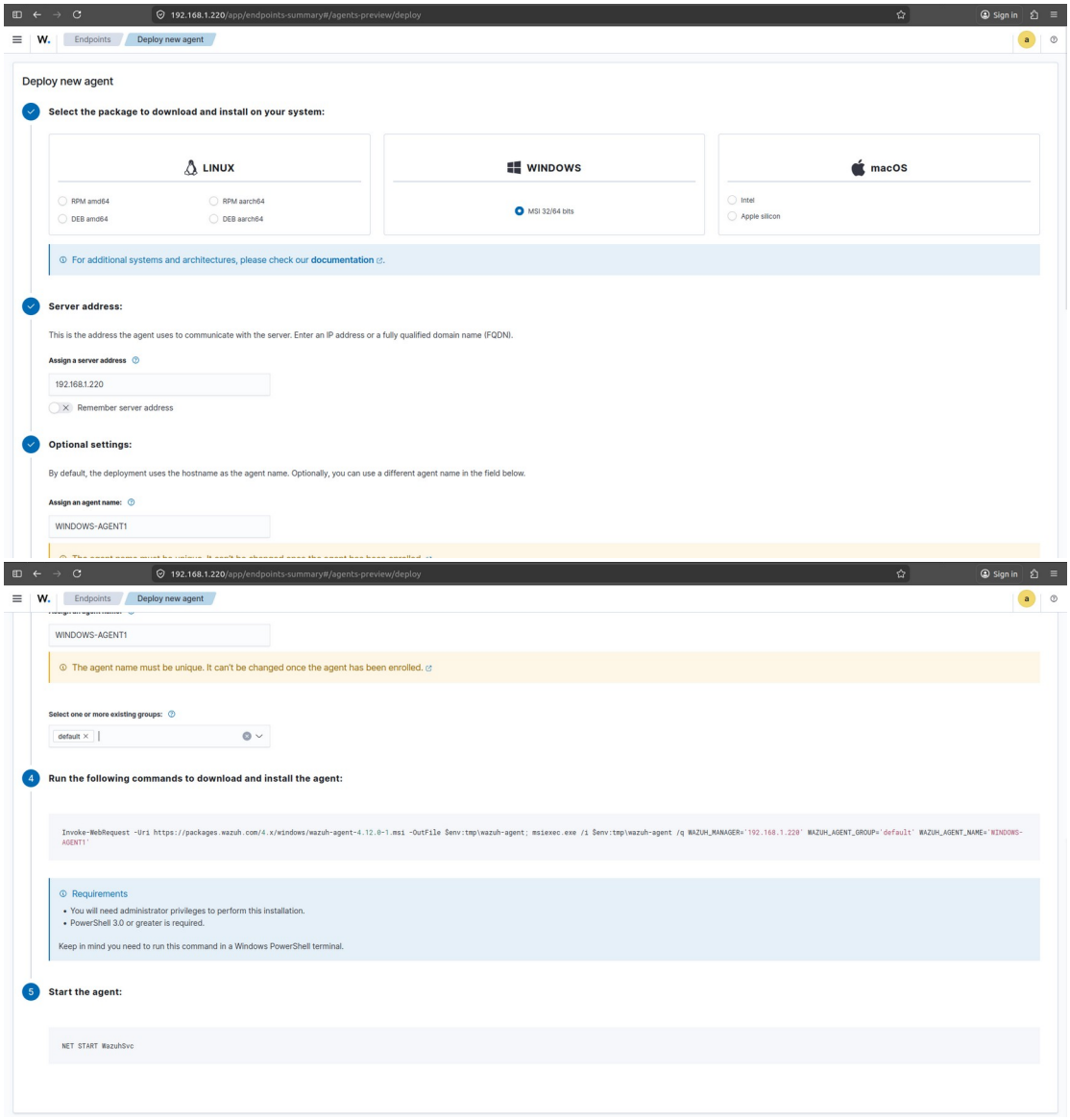


Wazuh is fetching data to display on the dashboard

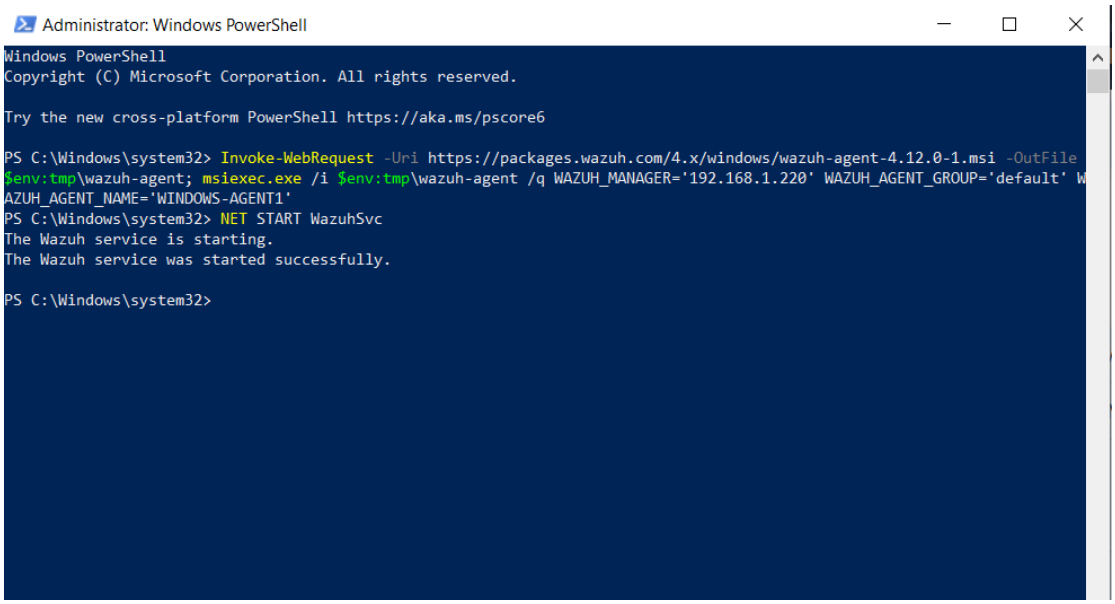




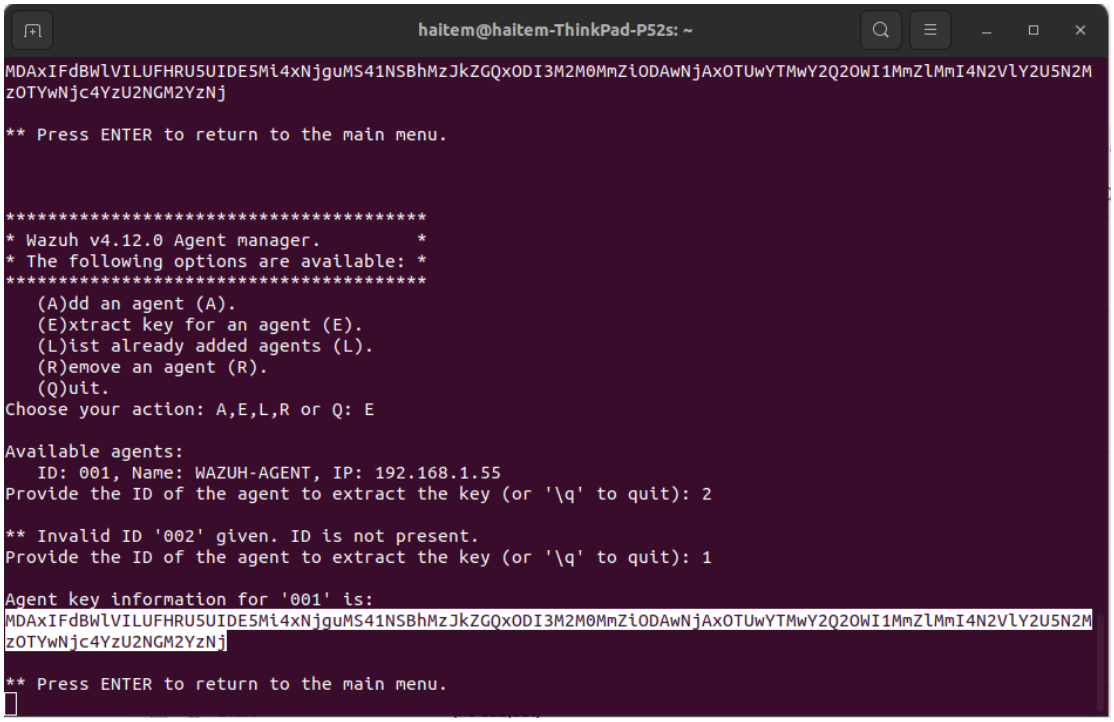
To deploy an agent, we go to deploy an agent and we follow the steps below, it will provide us with a command to run on powershell with admin privileges:

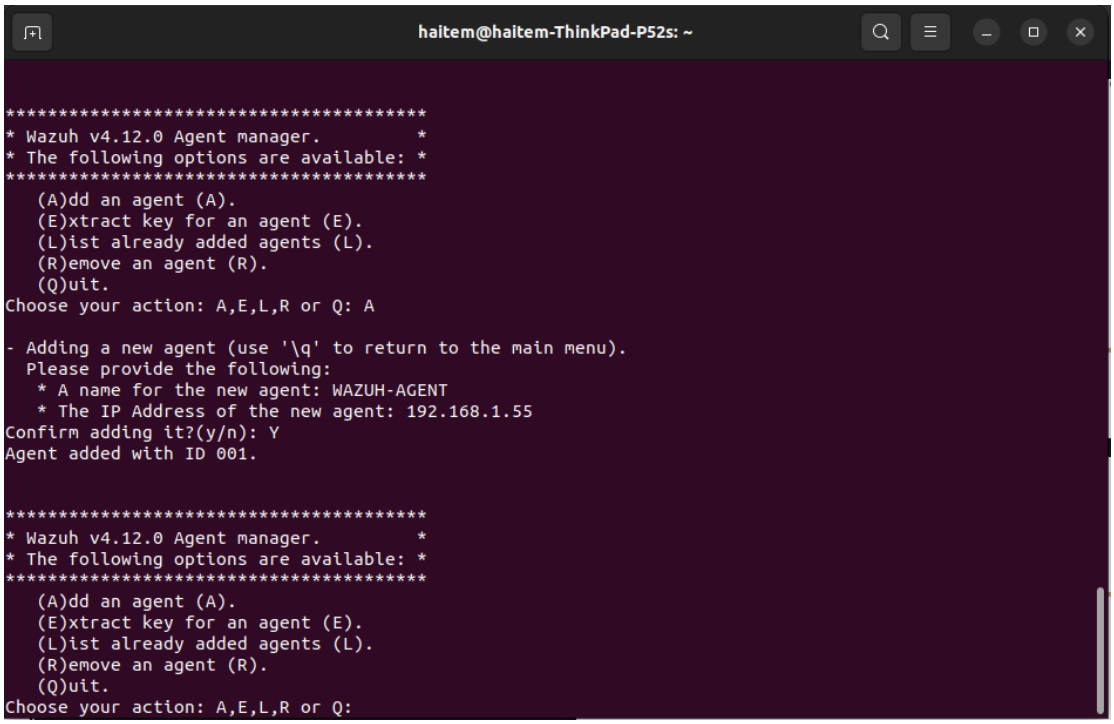


On the agent machine, we run on powershell the last command to install Wazuh:

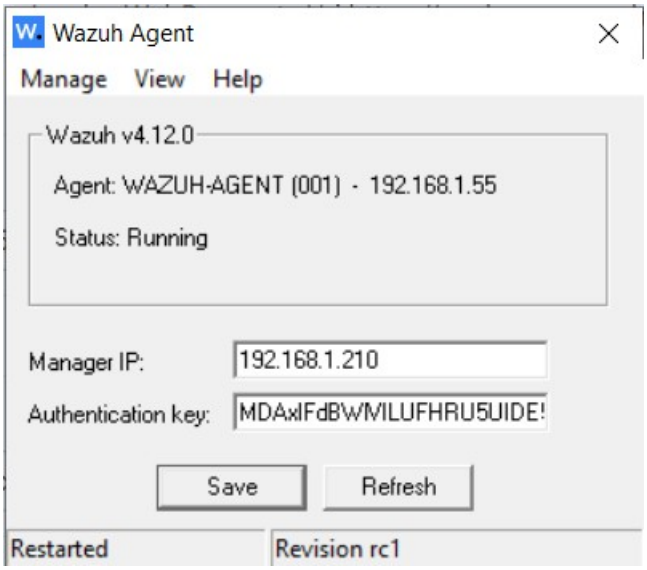


On the manager side we Extract the authentication key

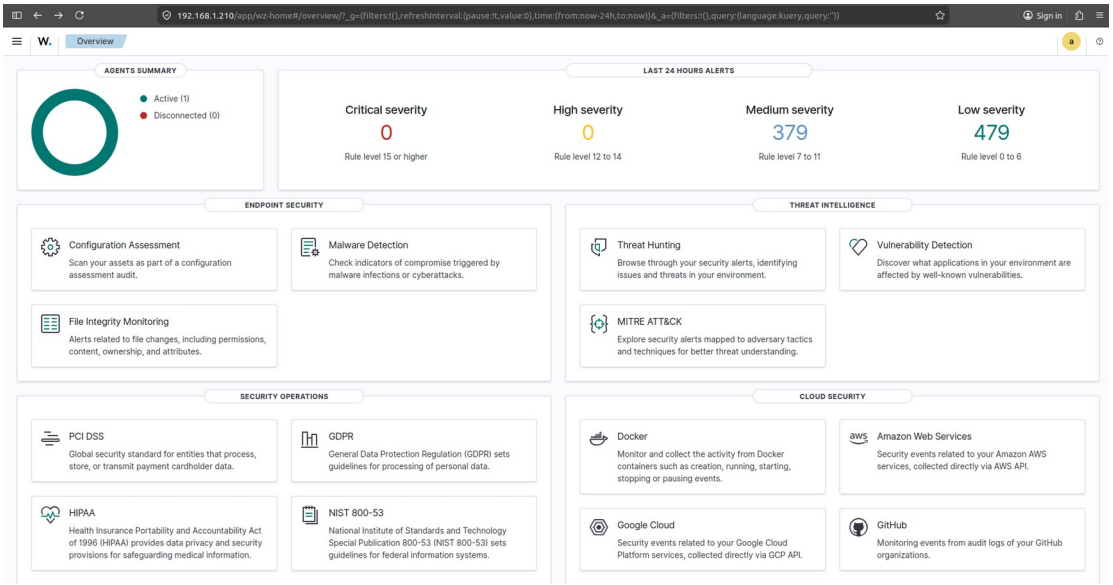


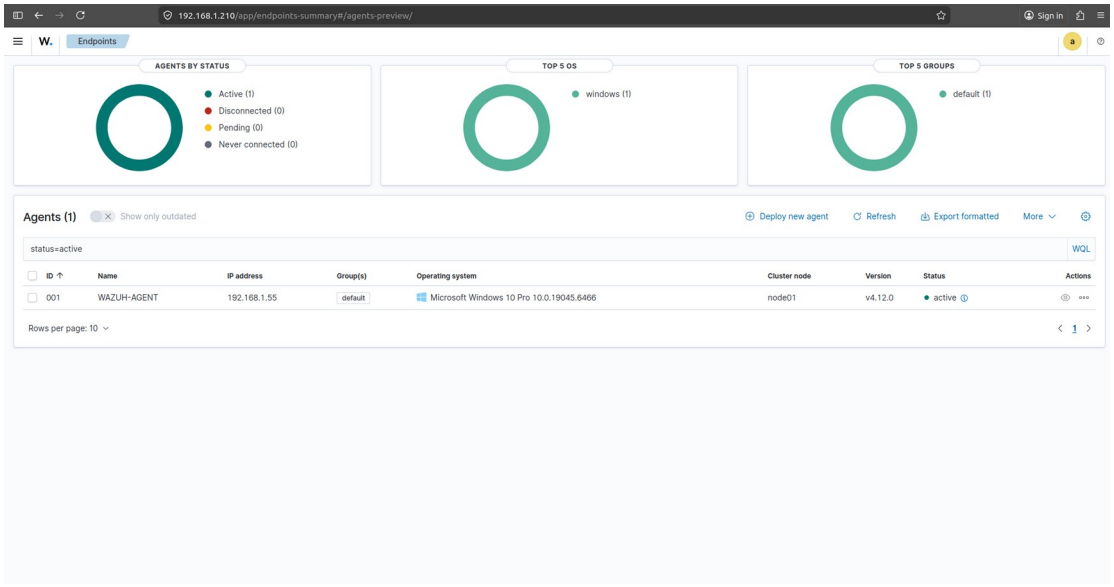


Agent enter auth key:

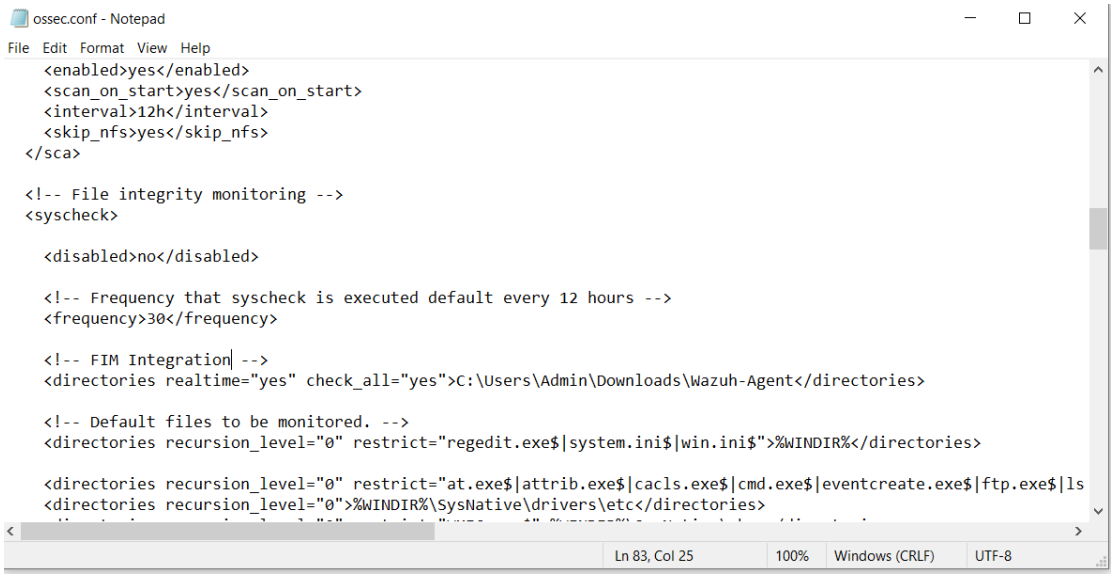


Now we can see the Agent is active and we can access the endpoint summary:

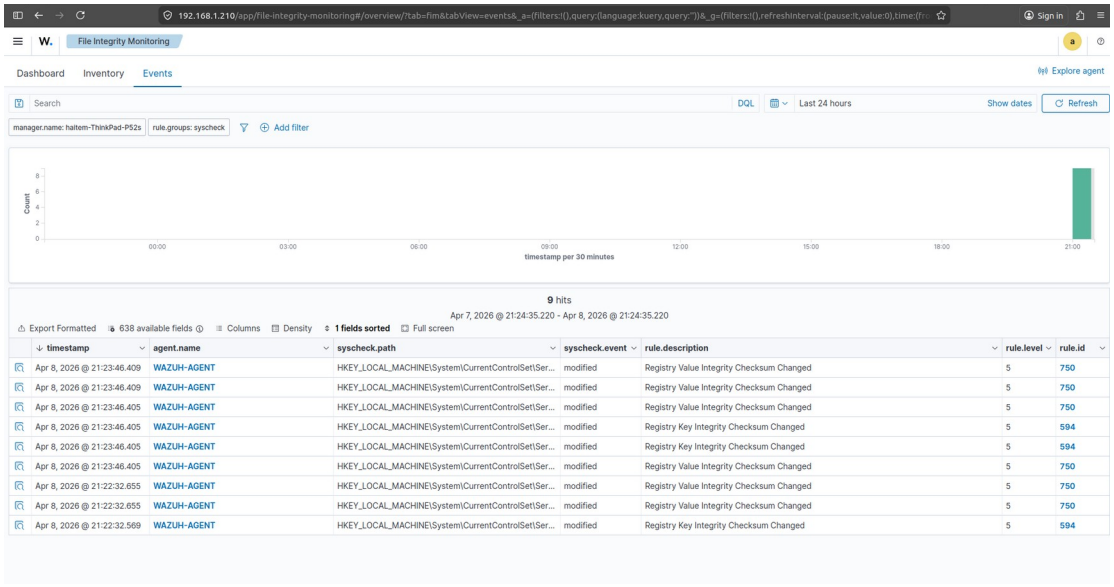




FIM configuration on the Agent



Event monitoring of the selected file



4.2 Detecting and Removing Malware Using Custom Scripts and Rules

To start the configuration, we will configure the ossec.config file, adding the command with the executable file that will be executed when detecting the malware:

```
<!-- COMMAND: Quarantine malicious file for customized script-->
<command>
  <name>win-quarantine-file</name>
  <executable>quarantine-file.exe</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<!-- Trigger file quarantine on FIM malware alert -->
<active-response>
  <command>win-quarantine-file</command>
    <location>defined-agent</location>
    <agent_id>002</agent_id>
  <rules_id>100002</rules_id>
  <timeout>0</timeout>
</active-response>

</ossec_config>
```

Then we add the rules to active respond based on the level of the threat, it can be extracted from the File Integration Monitoring page:

```
<group name="malware,syscheck,">

  <!-- Rule 1: Suspicious executable created in monitored directory -->
  <rule id="100001" level="12">
    <if_group>syscheck</if_group>
    <field name="file">\.exe$|\.bat$|\.cmd$|\.ps1$|\.com$</field>
    <description>Suspicious executable file created: $(file)</description>
    <mitre>
      <id>T1204</id>
    </mitre>
  </rule>

  <!-- Rule 2: EICAR test file or known malware hash detected -->
  <rule id="100002" level="15">
    <if_group>syscheck</if_group>
    <field name="file">eicar|malware|ransom</field>
    <description>Malware file detected in monitored directory: $(file)</description>
    <mitre>
      <id>T1204</id>
    </mitre>
  </rule>

  <!-- Rule 3: Mass file modification - ransomware behavior -->
  <rule id="100003" level="14" frequency="10" timeframe="30">
    <if_matched_group>syscheck</if_matched_group>
    <same_field>agent.id</same_field>
    <description>Possible ransomware: Mass file modification detected</description>
    <mitre>
      <id>T1486</id>
    </mitre>
  </rule>

</group>
```

Now, we create the quarantine-file.exe on the agent machine, a simple python script to remove the eicar malware :

```
import os
import glob

file_patterns = [
    r'C:\Users\Admin\Downloads\Wazuh-Agent\*.exe',
    r'C:\Users\Admin\Downloads\Wazuh-Agent\*.bat',
    r'C:\Users\Admin\Downloads\Wazuh-Agent\*.com',
    r'C:\Users\Admin\Downloads\Wazuh-Agent\eicar.*'
]

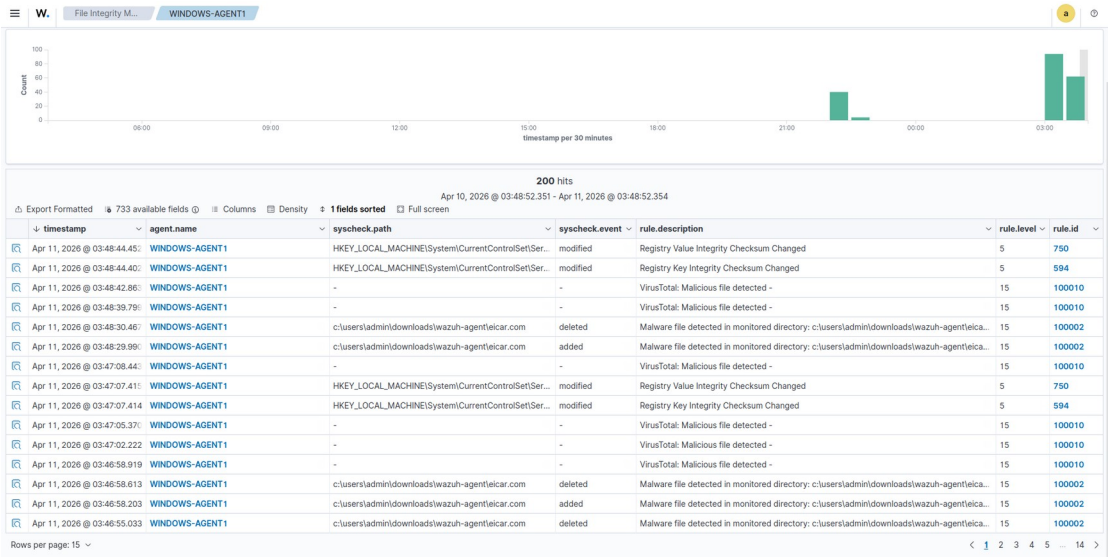
for pattern in file_patterns:
    # glob.glob finds all pathnames matching the pattern
    for file in glob.glob(pattern):
        try:
            os.remove(file)
            print(f"Deleted: {file}")
        except OSError as e:
            print(f"Error deleting {file}: {e}")
```

we tested the configuration by writing the following EICAR test file to the monitored folder, and we got the result on the dashboard:

```
PS C:\Program Files (x86)\ossec-agent\active-response\bin> Set-Content -Path "C:\Users\Admin\Downloads\Wazuh-Agent\eica
.com" -Value 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
```

The result:

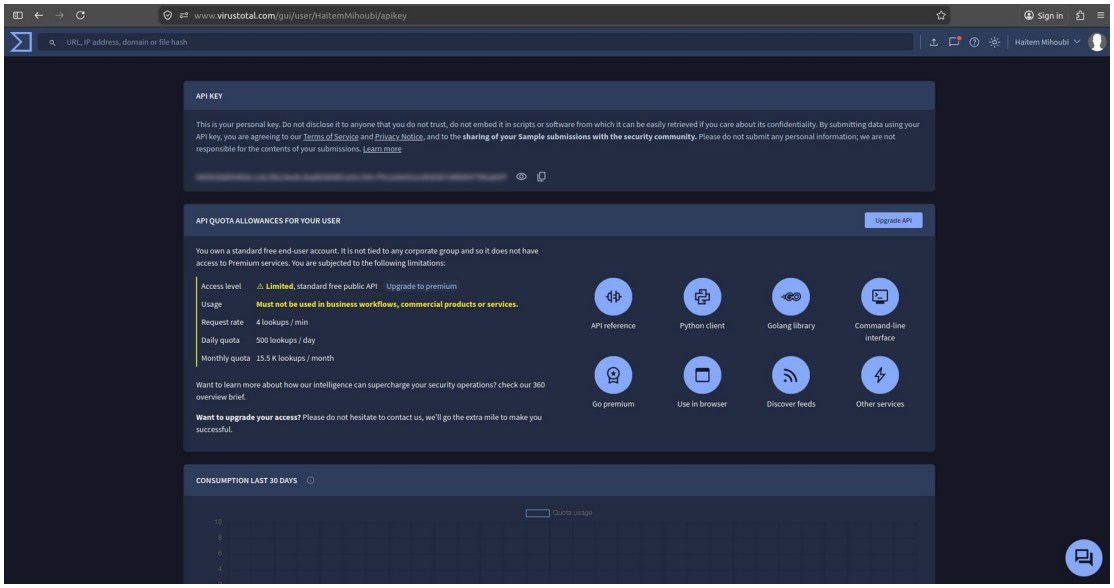
The logs on the dashboard show a response time of less than 1 second



4.3 Detecting and Removing Malware Using VirusTotal Integration

The script we've developed is very basic, creating a sophisticated one needs more effort and experience and can be a project itself. For that, we chose to use VirusTotal integration for more comprehensive active response solution.

First, we need to log in to the VirusTotal site, go to profile and extract the API Key:



On the server side, we integrate VirusTotal with Wazuh and set the active response command on /var/ossec/etc/ossec.conf targeting the Wazuh agent 001

```
<!-- VirusTotal Integration -->
  <integration>
    <name>virustotal</name>
    <api_key>06503d89468c1dc9b24edcdad656081e5c59cf01e8e62a36d387d0604796ab9f</api_key>
    <group>syscheck</group>
    <rule_id>550,553,554,100001,100002</rule_id>
    <alert_format>json</alert_format>
  </integration>

  <command>
    <name>remove-threat</name>
    <executable>remove-threat.exe</executable>
    <timeout_allowed>no</timeout_allowed>
  </command>

  <active-response>
    <disabled>no</disabled>
    <command>remove-threat</command>
    <location>defined-agent</location>
    <agent_id>001</agent_id>
    <rules_id>87105,100010,100011</rules_id>
  </active-response>

</ossec_config>
```

On /var/ossec/etc/rules/local_rules.xml, we set the rules for Wazuh to respond to threats:

```
<group name="local,syslog,sshd,syscheck,virustotal,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <!-- VirusTotal: File flagged as malicious -->
  <rule id="100010" level="15">
    <if_sid>87105</if_sid>
    <description>VirusTotal: Malicious file detected - $(file)</description>
    <mitre>
      <id>T1204</id>
    </mitre>
  </rule>

  <!-- VirusTotal: File flagged by multiple engines -->
  <rule id="100011" level="15">
    <if_sid>87105</if_sid>
    <field name="virustotal.positives">10</field>
    <description>VirusTotal: File flagged by 10+ AV engines - $(file)</description>
    <mitre>
      <id>T1204</id>
    </mitre>
  </rule>

  <!-- VirusTotal: File is clean -->
  <rule id="100012" level="3">
    <if_sid>87106</if_sid>
    <description>VirusTotal: File is clean - $(file)</description>
  </rule>

</group>
```

On the agent side, Python is already installed, so then we install PyInstaller and add it to global path:

```
PS C:\Windows\system32> pip install pyinstaller
Requirement already satisfied: pyinstaller in c:\users\admin\appdata\roaming\python\python313\site-packages (6.19.0)
Requirement already satisfied: altgraph in c:\users\admin\appdata\roaming\python\python313\site-packages (from pyinstall
er) (0.17.5)
Requirement already satisfied: packaging>=22.0 in c:\users\admin\appdata\roaming\python\python313\site-packages (from py
installer) (26.0)
Requirement already satisfied: pefile>=2022.5.30 in c:\users\admin\appdata\roaming\python\python313\site-packages (from
pyinstaller) (2024.8.26)
Requirement already satisfied: pyinstaller-hooks-contrib>=2026.0 in c:\users\admin\appdata\roaming\python\python313\site
-packages (from pyinstaller) (2026.4)
Requirement already satisfied: pywin32-ctypes>=0.2.1 in c:\users\admin\appdata\roaming\python\python313\site-packages (f
rom pyinstaller) (0.2.3)
Requirement already satisfied: setuptools>=42.0.0 in c:\users\admin\appdata\roaming\python\python313\site-packages (from
 pyinstaller) (82.0.1)
```

```
PS C:\Windows\system32> $env:PATH += ";C:\Users\Admin\AppData\Roaming\Python\Python313\Scripts"
PS C:\Windows\system32> pyinstaller --version
85 DEPRECATION: Running PyInstaller as admin is not necessary nor sensible. Run PyInstaller from a non-administrator ter
minal. PyInstaller 7.0 will block this.
ERROR: Do not run pyinstaller from C:\Windows\system32. cd to where your code is and run pyinstaller from there. Hint: Y
ou can open a terminal where your code is by going to the parent folder in Windows file explorer and typing cmd into the
 address bar.
PS C:\Windows\system32> [Environment]::SetEnvironmentVariable("PATH", $env:PATH + ";C:\Users\Admin\AppData\Roaming\Pytho
n\Python313\Scripts", "User")
```

We copy the remove-threat script from the Wazuh documentation and add it to ossec/active-response/bin

remove-threat - Notepad

File Edit Format View Help

```
# Copyright (C) 2015-2025, Wazuh Inc.
# All rights reserved.

import os
import sys
import json
import datetime
import stat
import tempfile
import pathlib

if os.name == 'nt':
    LOG_FILE = "C:\\Program Files (x86)\\ossec-agent\\active-response\\active-responses.log"
else:
    LOG_FILE = "/var/ossec/logs/active-responses.log"

ADD_COMMAND = 0
DELETE_COMMAND = 1
CONTINUE_COMMAND = 2
ABORT_COMMAND = 3

OS_SUCCESS = 0
OS_INVALID = -1

class message:
    def __init__(self):
        self.alert = ""
        self.command = 0

def write_debug_file(ar_name, msg):
    with open(LOG_FILE, mode="a") as log_file:
        log_file.write(str(datetime.datetime.now().strftime('%Y/%m/%d %H:%M:%S')) + " " + ar_name + ": " + ms

def setup_and_check_message(argv):
```

Ln 1, Col 1 | 100% | Windows (CRLF) | UTF-8

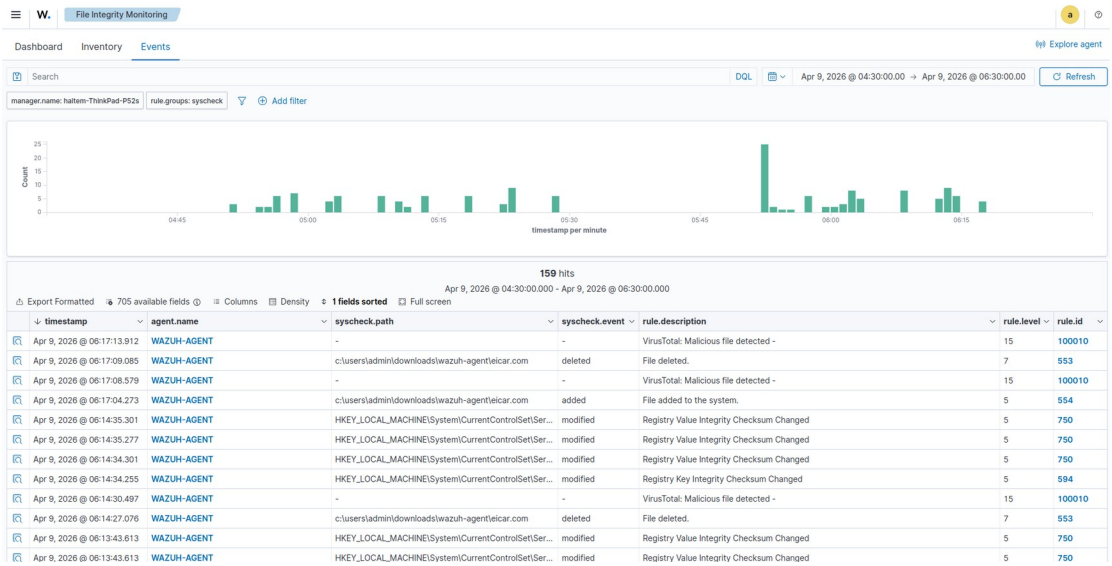
Then we use pyinstaller to make the executable file from the python script:

```
PS C:\Program Files (x86)\ossec-agent\active-response\bin> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Force
PS C:\Program Files (x86)\ossec-agent\active-response\bin> pyinstaller -F C:\path\to\remove-threat.py
84 DEPRECATION: Running PyInstaller as admin is not necessary nor sensible. Run PyInstaller from a non-administrator ter
minal. PyInstaller 7.0 will block this.
298 INFO: PyInstaller: 6.19.0, contrib hooks: 2026.4
299 INFO: Python: 3.13.5
334 INFO: Platform: Windows-10-10.0.19045-SP0
335 INFO: Python environment: C:\Python313
```

After restarting both side and turn off real-time protection of Microsoft Defender, Server and Agent we are now ready for a small test. We wrote the following EICAR test file to the monitored folder:

```
PS C:\Program Files (x86)\ossec-agent\active-response\bin> Set-Content -Path "C:\Users\Admin\Downloads\Wazuh-Agent\eica
.com" -Value 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
```

And we can see the result on the Wazuh dashboard:



The logs on the dashboard show a delay of 5–9 seconds from creation to deletion because of API access. In real-world, it is more but in our controlled environment it is manageable, VirusTotal offers accuracy over speed

5. Evaluation & Results

5.1 Methodology

The evaluation measures the *Mean Time to Respond* (MTTR), defined here as the wall-clock interval between the creation of a malicious file on the monitored endpoint and the moment at which that file is deleted by the active-response action. Three response configurations were measured against an identical attack scenario: (a) custom rule with local quarantine script (§4.2), (b) custom rule with VirusTotal integration (§4.3), and (c) manual response by an analyst using the Windows GUI as a baseline.

Each configuration was tested over multiple trials. For each trial, the EICAR test file was written into the monitored Sensitive Folder via the attack vector described in §5.2, and the timestamps for file creation and file deletion were extracted from the corresponding *syscheck* and *active-response* records in the Wazuh dashboard. MTTR was computed as the arithmetic mean of these intervals.

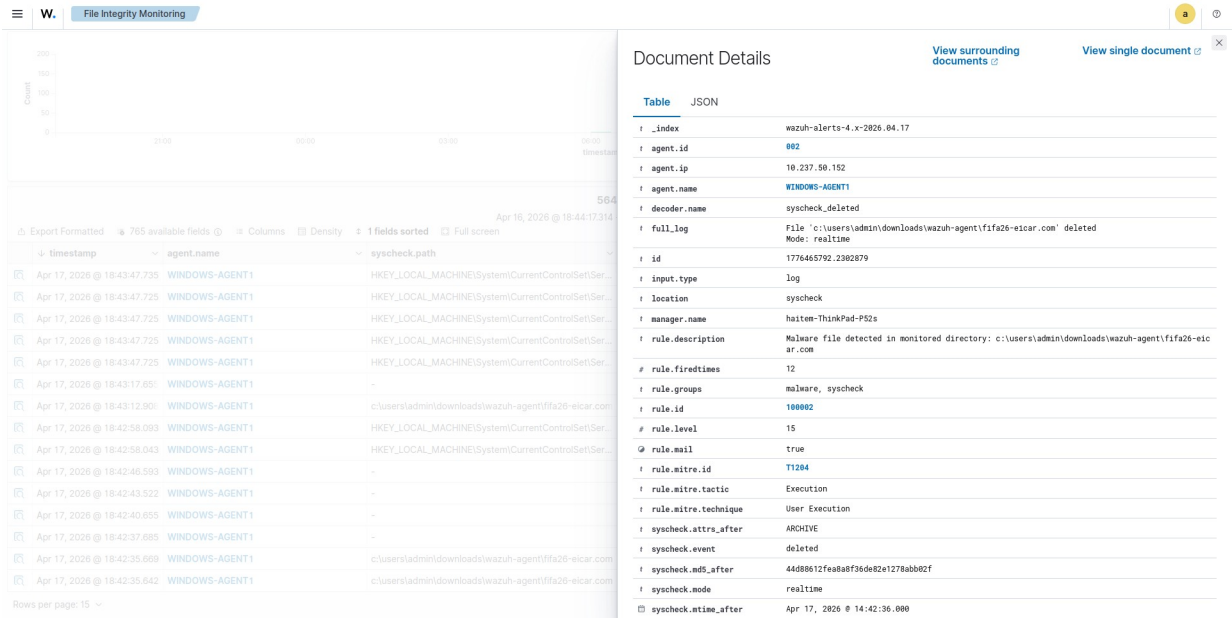
5.2 Attack Scenario — Phishing-Driven Malware Download

To make the evaluation realistic rather than synthetic, the attack was delivered through a simulated phishing scenario. A web server was hosted on a separate Kali Linux machine on the lab network, and a convincing landing page was constructed using a 2026 FIFA World Cup theme, chosen to combine topical relevance with a plausible sense of urgency. The page invited the visitor to download a “specialized application for real-time tournament updates”; the download served the EICAR test file.

The HTML and styling for the landing page were generated with the assistance of an AI coding assistant to accelerate the social-engineering scaffolding. This is consistent with the broader observation that defenders and attackers now have access to similar AI capabilities for peripheral attack tasks (Section 2.1), and it allowed the project’s effort to remain focused on the detection-and-response pipeline itself rather than on web design.

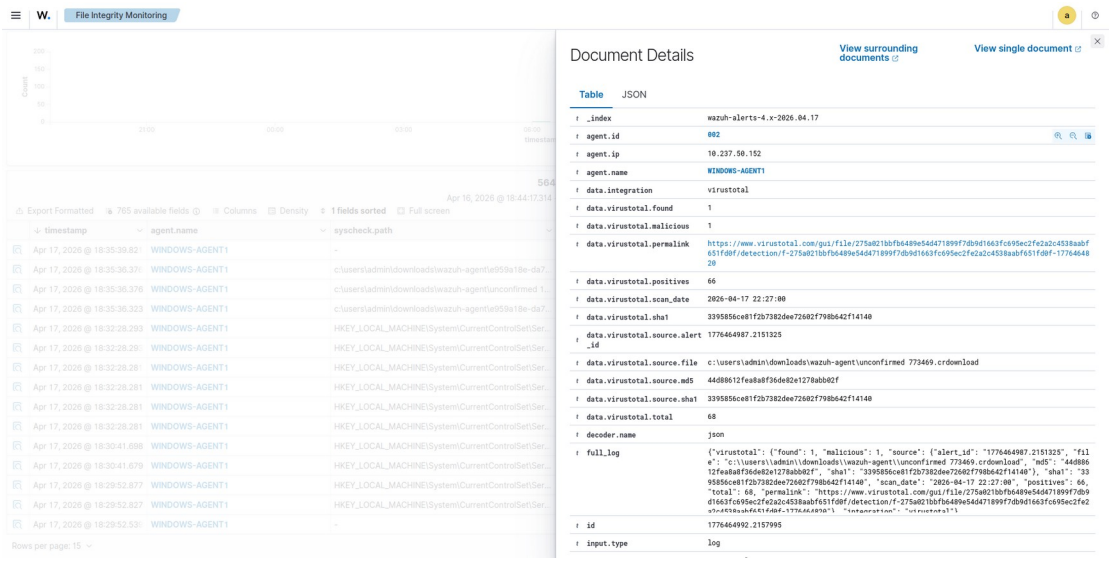
The victim machine was the Wazuh-monitored Windows 10 endpoint. Once the download completed and the EICAR file was written into the Sensitive Folder, the FIM detection-and-response pipeline was triggered.

5.3 Results: Custom Rules and Scripts



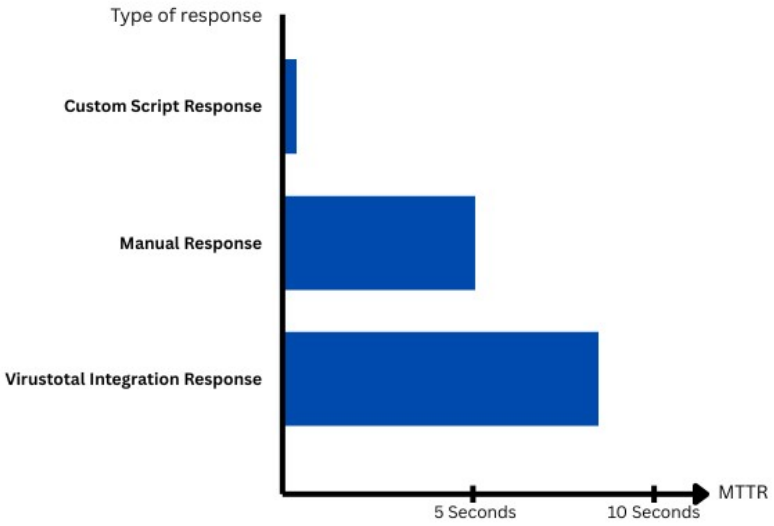
Based on the implementation data and this test, the Mean Time to Respond (MTTR) is less than 1 second.

5.4 Results: VirusTotal Integration



Based on the implementation data and this test, the Mean Time to Respond (MTTR) ranges between 5 and 9 seconds.

5.5 Comparative Analysis



The evaluation reveals a significant performance gap between the custom response script and VirusTotal integration. The VirusTotal module relies on external API calls to compare file hashes against its database, introducing latency due to internet dependency. In contrast, the custom script executes locally for near-instantaneous action, though it lacks the sophisticated detection depth of the integration.

We have also tested manual response, deleting the malware via the GUI took approximately 5 seconds. While functional for a single instance, this approach is impractical and inefficient for scaling across dozens of machines.

We conclude that while the custom script provides a faster response time, it is less accurate than the VirusTotal integration; furthermore, manual response fails to meet the operational demands of a multi-endpoint environment.

6. Discussion

6.1 Limitations

Despite demonstrating a functional and measurable automated response capability, the current implementation operates under several constraints that limit the generalizability and robustness of its security guarantees:

Rule-Based Detection Coverage: The system's effectiveness is strictly dependent on the quality and completeness of its XML rule-set and response scripts. Threats that do not generate file system or log events matching an existing Wazuh decoder will fail to trigger an alert or a response.

Agent Availability as a Single Point of Failure: The detection and response pipeline depends entirely on the Wazuh Agent remaining operational on the Windows 10 endpoint. An adversary who successfully disables or disconnects the agent renders the Manager blind to all subsequent activity on that host.

Controlled Lab Environment: The evaluation was conducted in an isolated lab environment rather than a production network. Real-world deployments introduce additional complexities, such as heterogeneous endpoint configurations, high volumes of benign activity that may produce false positives, network latency, and interference from concurrent security tools.

6.2 Future Work

Machine Learning Integration: Incorporating a machine learning component for anomaly detection would extend coverage beyond the boundaries of the existing static rule-set, allowing for the identification of previously unknown threats.

Scalability Testing: Expanding the evaluation from a two-node lab environment to a multi-endpoint deployment would validate the system's performance, false-positive rate, and operational manageability under conditions closer to those faced by a modern enterprise.

7. Conclusion

This project demonstrates that professional-grade, automated threat detection and response is achievable within the resource constraints of small-to-medium enterprises, using open-source tooling and without the operational overhead of a dedicated Security Operations Center. The result is a fully functional, end-to-end security pipeline built on Wazuh that detects unauthorized file system activity on a Windows 10 endpoint, and autonomously executes a remediation action without human intervention.

The core finding of the evaluation is that the system compresses the response window from the minutes required for manual intervention to a sub-second automated execution, demonstrating a substantial and measurable reduction in Mean Time to Respond using the custom rules and script solution and high accuracy rate using VirusTotal Integration. This validates the central premise of the project: that the latency bottleneck in SME security is not fundamentally a resource problem, but a configuration and automation problem and that open-source tools, properly deployed, are sufficient to address it.

Beyond the performance result, the project makes a broader architectural argument. The three-stage Detection–Decision–Response pipeline introduced here provides a reusable template for automated security response that is not specific to Wazuh or to the particular threat scenarios evaluated. The same structural pattern is applicable across a wide range of endpoint threats and deployable in a variety of organizational contexts.

The practical implication of this work is straightforward. An SME with physical machines and the Wazuh documentation can deploy a system that autonomously detects and responds to a meaningful class of real-world endpoint threats. The barrier to entry for enterprise-grade automated security has never been lower. This project demonstrates not only that the barrier is surmountable, but exactly how to surmount it.

References

Craig. (2023, August 10). *Microsoft Sentinel vs. Google Chronicle.* Craig's Cloud IT Pro. https://craigclouditpro.wordpress.com/2023/08/10/microsoft-sentinel-vs-google-chronicle/

FMI Security. (n.d.). *Wazuh vs. OSSEC, Graylog, and ELK Stack: A real-world comparison.* FMI Security News. https://news.fmisec.com/02-wazuh-vs-ossec-graylog-and-elk-stack-a-real-world-comparison

Hammond, J. (2024). *Wazuh tutorial* [Video]. YouTube. https://www.youtube.com/watch?v=vJZAVZOIpfA

Microsoft Security. (n.d.). *What is a security operations center (SOC)?* Microsoft. https://www.microsoft.com/en-us/security/business/security-101/what-is-a-security-operations-center-soc

Rebello, R. (2025). *WAZUH home lab: SIEM and file integrity monitoring* [Document]. Google Drive. https://drive.google.com/file/d/1CDwjb5nSMaP73q2W-znWQpI6523oPxlI/view

Srua, S. (2024). *Exercise caution: This lab involves working with potentially malicious files.* Medium. https://medium.com/@srua2830/exercise-caution-this-lab-involves-working-with-potentially-malicious-files-c359094044ae

Unihackers. (n.d.). *SIEM comparison: Splunk, QRadar, and Sentinel.* Unihackers Blog. https://unihackers.com/blog/siem-comparison-splunk-qradar-sentinel

V500 Systems. (n.d.). *False positive, false negative, true positive, and true negative.* V500 Systems. https://www.v500.com/false-positive-false-negative-true-positive-and-true-negative/

VirusTotal. (n.d.). *VirusTotal* [Online service]. https://www.virustotal.com/gui/home/upload

Wazuh, Inc. (n.d.-a). *Architecture: Getting started with Wazuh.* Wazuh Documentation. https://documentation.wazuh.com/current/getting-started/architecture.html

Wazuh, Inc. (n.d.-b). *Wazuh agent: Components.* Wazuh Documentation. https://documentation.wazuh.com/current/getting-started/components/wazuh-agent.html

Wazuh, Inc. (n.d.-c). *Active response: Capabilities.* Wazuh Documentation. https://documentation.wazuh.com/current/user-manual/capabilities/active-response/index.html

Wazuh, Inc. (n.d.-d). *Incident response: Use cases.* Wazuh Documentation. https://documentation.wazuh.com/current/getting-started/use-cases/incident-response.html

Wazuh, Inc. (n.d.-e). *Detecting and responding to malicious files using CDB lists and active response.* Wazuh Blog. https://wazuh.com/blog/detecting-and-responding-to-malicious-files-using-cdb-lists-and-active-response/

Wazuh, Inc. (n.d.-f). *File integrity monitoring: Capabilities.* Wazuh Documentation. https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html

Wazuh, Inc. (n.d.-g). *Wazuh agent package: Windows installation guide.* Wazuh Documentation. https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html

Wazuh, Inc. (n.d.-h). *Default active response scripts: Capabilities.* Wazuh Documentation. https://documentation.wazuh.com/current/user-manual/capabilities/active-response/default-active-response-scripts.html

Zaid, T., & Garai, S. (2024). Emerging trends in cybersecurity: A holistic view on current threats, assessing solutions, and pioneering new frontiers. *Blockchain in Healthcare Today, 7*. https://doi.org/10.30953/bhty.v7.302